

SpringSecurity



P.Mathieu

IUT de Lille
<http://www.iut-a.univ-lille.fr>
prenom.nom@univ-lille.fr

Plan



Rappels cryptographiques

Spring Security

Le Bean `securityFilterChain`

Le Bean `userService`

Protection contre l'attaque CSRF

Conclusion

Rappels cryptographiques

Rappels cryptographiques : I



L'encodage

- ▶ Objectif : coder différemment (par exemple pour les accents ou pour éviter des caractères spéciaux.)
- ▶ **Aucune volonté de dissimuler**
- ▶ En général passer du binaire à l'ascii
- ▶ Ex : `Base64`, `uuEncode`, `binHex`
- ▶ Usage : échange des tokens d'identification entre client et serveur
- ▶ La compression (RLE, ZIP, MP3, JPEG) est une forme d'encodage

Rappels cryptographiques

Rappels cryptographiques : II



Le chiffrement

- ▶ Objectif : masquer une donnée tout en permettant son déchiffrement
- ▶ **Utilisation d'une clé**
- ▶ Ex : Caesar, Vigenère, Enigma
- ▶ Deux grandes techniques :
 - ▶ symétriques (même clé pour chiffrer et déchiffrer) : DES, AES
 - ▶ asymétriques (clés différentes) : RSA, DSA
- ▶ Usage : chiffrement de la communication en client et serveur (SSL)

Rappels cryptographiques

Rappels cryptographiques : III

Le hashage

- Fournit une **empreinte de taille fixe**
(bien plus petite que la donnée initiale)
- L'empreinte ne contient pas la donnée initiale
c'est donc **irréversible**
- Ex : MD5 (128bits), Sha-1 (160bits), Sha-256 (256bits), Sha-512
- Pb : Deux données différentes peuvent avoir la même empreinte (collision)
→ introduction du sel
- Ex avec sel : SCrypt, PBKDF2, BCrypt, Argon2
- Usage : stockage des mdp des utilisateurs

Rappels cryptographiques

Les propriétés d'une bonne fonction de hashage

- **Déterminisme** : si le message de départ est le même, alors le résultat doit être le même.
- **Efficacité** : le calcul doit être rapide
- **Non inversible** : il doit être impossible de recréer le message d'origine à partir de l'empreinte
- **Aisance de distinction** : deux messages extrêmement similaires doivent générer deux résultats très différents.
- **Unicité des résultats** : il doit être impossible de générer deux résultats identiques avec deux messages différents.

Rappels cryptographiques

Les différents types d'attaques

- Par dictionnaire : tester les mots de passe usuels de la langue :
123456, azerty, password, 12345, bonjour, 123123, admin,
les prénoms usuels etc ...
- Par force brute : tester des mots de passe au hasard, par exemple les mots
de 8 chiffres ou lettres (8^{36})
(D'où introduction de méthodes à facteur de coût)
- Rainbow tables : tables de chaînes d'empreintes
(D'où introduction de sel unique pour chaque mdp)

Rappels cryptographiques

Fonctions de hashage utilisées dans les crypto-monnaies

- sha-256 : Bitcoin (BTC), Mastercoin(MSC), et tous les fork Bitcoin
- ethash : Ethereum (ETH), Metaverse Entropy (ETP)
- Scrypt (avec sel) : Litecoin (LTE), Dogecoin (DOGE), Auroracoin (AUR)
Coinye (COINYE)
- cryptonight : Dero (DERO)
- randomX : Monero (XMC)

Remarque : Des monnaies comme Polkadot et Cardano utilisent des méthodes de consensus (PoS)

Rappels cryptographiques

Un exemple avec BCrypt

Empreinte BCrypt

```
$2y$10$gMOWsvhvArI10Y1GZYm23.1Uw470qBxYV/B5Cj.WzbX5R2nAZ2.0i
```

- ▶ test avec mot de passe = *blanche neige et les 7 nains*
- ▶ 2y identifie la version de l'algorithme bcrypt qui a été utilisé.
- ▶ 10 est le facteur coût utilisé (10 par défaut en PHP, paramétrable en SpringBoot (16 par défaut)).
- ▶ puis sel et empreinte, concaténés et encodés en Base64
 - ▶ Les 22 premiers caractères correspondent à une valeur sur 128 bits pour le sel.
 - ▶ Les caractères restants correspondent à l'empreinte

Plan

Rappels cryptographiques

Spring Security

Le Bean `securityFilterChain`

Le Bean `userService`

Protection contre l'attaque CSRF

Conclusion

Spring Security

Kézako ?

`SpringSecurity` est une collection de `Filter` et d'encodeurs qui aident à l'authentification puis l'autorisation d'accès aux différentes pages d'une application.

`SpringSecurity` permet de répondre à quatre questions :

- ▶ Comment sont codés les mots de passe ?
- ▶ Où sont stockés les login / mots de passe ?
- ▶ Quelle technique d'authentification est utilisée ?
- ▶ Quelles pages doivent être protégées et pour quels rôles ?

Spring Security

Principe

- ▶ Version actuelle 7.0.1 – Documentation [ici](#)
- ▶ `SpringSecurity` sécurise le dialogue entre l'application et l'utilisateur
- ▶ Ce dialogue est géré par la `DispatcherServlet`
- ▶ L'objectif de `SpringSecurity` est d'insérer des filtres dans ce dialogue
- ▶ Il y a des dizaines de filtres gérés par `SpringSecurity`
`CsrfFilter`, `LogoutFilter`, `BasicAuthenticationFilter`, ...

Ces filtres sont paramétrables par des beans et sont ordonnés par le Framework

Spring Security

Deux étapes fondamentales



Dans cette chaîne de filtres il y a deux étapes impératives :

► L'authentification

Est-ce que le login/mdp est reconnu ?

Ceci est géré par le bean `UserDetailsService`

Si non authentifié, Spring renvoie une erreur 401 `unauthorized`

► L'autorisation

Est-ce que l'utilisateur authentifié a le droit d'accéder à la page

Ceci est géré par le bean `SecurityFilterChain`

Si non autorisé, Spring renvoie une erreur 403 `Forbidden`

La `FilterChain` s'occupe des deux, et si tout est ok, passe la requête à la `DispatcherServlet`

Spring Security

Le comportement par défaut



- Dès que la dépendance est mise dans le pom ...

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

- Un Utilisateur `user` est créé en mémoire (mode `inMemory`)

- Un password est affiché dans la console

Using generated security password: 82b113cb-46c3-4651-a5f3-a30bee54bded

- Toutes les pages sont alors sécurisées et nécessitent login/mdp (mode `anyRequest().authenticated()`)

- Un formulaire de login est généré par SpringBoot (mode `formLogin`)

Spring Security

Affinement du comportement ...



- Changer le login/mdp de l'utilisateur par défaut

Dans le fichier `application.properties` mettre

```
spring.security.user.name=paul
spring.security.user.password=paul
spring.security.user.role=admin
```

- Pour aller encore plus loin implémenter l'un ou les deux beans `SecurityFilterChain` et `UserDetailsService`

Spring appelle lui même la page de login dès que l'utilisateur essaye d'accéder à une page protégée

Spring Security

Un exemple ... qui fournirait la même chose que la configuration par défaut du T14



```
@Configuration
public class Security {
    @Bean
    public SecurityFilterChain mesautorisations(HttpSecurity http)
        throws Exception {
        return http.authorizeHttpRequests((authorize) -> authorize
            .dispatcherTypeMatchers(DispatcherType.FORWARD).permitAll() // pour les vues
            .anyRequest().authenticated()
        )
        .formLogin(Customizer.withDefaults())
        .build();
    }

    @Bean
    public UserDetailsService mesutilisateurs() {
        UserDetails user1 = User.builder()
            .username("user")
            .password("{bcrypt}$2y$10$gMOWsvhvArI10YlGZym23.1Uw470qBxVv/B5Cj.WzbX5R2nA22.0i")
            .roles("ADMIN")
            .build();
        return new InMemoryUserDetailsManager(user1);
    }
}
```

Par défaut Spring sait vérifier les hash les plus fréquents (noop,MD5,bcrypt,...)

Plan

Rappels cryptographiques

Spring Security

Le Bean securityFilterChain

Le Bean userDetailsService

Protection contre l'attaque CSRF

Conclusion

Le Bean securityFilterChain

Le filtrage des requêtes

`authorizeHttpRequests` permet de filtrer les requêtes et leur appliquer, dans l'ordre, des contraintes d'accessibilité

Filtrage :

- ▶ `requestMatchers("/", "/public", "voir/**", "lister/**")`
- ▶ `requestMatchers("/user/{userName}")`
- ▶ `anyRequest()`

Contraintes :

- ▶ `authenticated()`
- ▶ `permitAll()`
- ▶ `hasAuthority("USER")` ou `hasAnyAuthorities(...)`
- ▶ `hasIpAddress(...)`
- ▶ `denyAll()`

Le Bean securityFilterChain

Bannières de login

Trois possibilités :

- ▶ Gérée par le navigateur (token basic) : `httpBasic(...)`
(plutôt fait pour le stateless, type services REST)
- ▶ Gérée par le serveur (token session) : `formLogin(...)`
(plutôt pour sites MVC ... avec Vue)
- ▶ Page personnalisée : `formLogin(...)` + `loginPage(url)`

En cas de `formLogin`, le end-point `/logout` est automatiquement généré

Le Bean securityFilterChain

Un exemple plus complet

```
@Bean
public SecurityFilterChain mesautorisations(HttpSecurity http)
    throws Exception {
    return http.authorizeHttpRequests((authorize) -> authorize
        .dispatcherTypeMatchers(DispatcherType.FORWARD, DispatcherType.ERROR).permitAll()
        .requestMatchers("/").permitAll()
        .requestMatchers("/admin").hasAuthority("ADMIN")
        .anyRequest().authenticated()
    )
    // .httpBasic( Customizer.withDefaults() )
    .formLogin( configurer ->
        configurer.defaultSuccessUrl("/xxx",true) // true = toujours redirect
        .failureUrl("http://xxx?error")
    )
    .logout( configurer ->
        configurer.logoutSuccessUrl("/xxx")
    )
    .rememberMe( configurer ->
        configurer.rememberMeParameter("remember-me")
        .useSecureCookie(true)
    )
    .build();
}
```

Plan

Rappels cryptographiques

Spring Security

Le Bean `securityFilterChain`

Le Bean `userDetailsService`

Protection contre l'attaque CSRF

Conclusion

Le Bean `userDetailsService`

Les encodeurs

Spring fournit un certain nombre d'encodeurs "standards" :

- ▶ `Pbkdf2PasswordEncoder()` ; pour pbkdf2
- ▶ `BCryptPasswordEncoder()` ; pour bcrypt
- ▶ `SCryptPasswordEncoder()` ; pour scrypt

Pour changer d'encodeur il faut injecter un bean de type `PasswordEncoder` :

```
@Bean
public PasswordEncoder encoder() {
    return new BCryptPasswordEncoder();
}
```

Le Bean `userDetailsService`

Le même exemple qu'au T16 ... mais avec un encodeur

```
@Configuration
public class Security {
    @Bean
    public SecurityFilterChain mesautorisations(HttpSecurity http)
        throws Exception {
        return http.authorizeHttpRequests((authorize) -> authorize
            .anyRequest().authenticated()
        )
        .formLogin(Customizer.withDefaults())
        .build();
    }

    @Bean
    public UserDetailsService mesutilisateurs() {
        UserDetails user1 = User.builder()
            .username("user")
            .password(encoder().encode("password"))
            .roles("ADMIN")
            .build();
        return new InMemoryUserDetailsManager(user1);
    }

    @Bean
    public PasswordEncoder encoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Le Bean `userDetailsService`

On peut bien sûr écrire son propre encodeur

Chaque encodeur doit implémenter 2 méthodes : `encode` et `matches`

L'exemple d'un NoOp password encoder

```
public class MonPasswordEncoder implements PasswordEncoder {
    @Override
    public String encode(CharSequence charSequence) {
        // encode l'entrée et renvoie la valeur encodée
        return charSequence;
    }

    @Override
    public boolean matches(CharSequence charSequence, String s) {
        // utilisée par l'authentifier pour vérifier le mdp
        return encode(charSequence).equals(s);
    }
}
```

Le Bean userDetailsService

Un outil remarquable : Le DelegatingPasswordEncoder



```
PasswordEncoderFactories.createDelegatingPasswordEncoder();
```

- ▶ Il encode en BCrypt,
- ▶ il match tous les encodages (bcrypt, pbkdf2, scrypt, MD4, MD5, noop, SHA-1, SHA-256) selon le préfixe du mot de passe

username	password	enabled
paul	{noop}paul	t
phil	{noop}phil	t
jean	{MD5}b71985397688d6f1820685dde534981b	t
mathieu	{bcrypt}\$2a\$10\$INHV3N2mRpWRyvvGAOYZeeNXKghHK01hwW4alYu46zU8ma71qTenW	t

Permet de faire évoluer le chiffage vers de nouveaux encodeurs plus facilement
C'est le "matcher" par défaut depuis Spring Security V5

Le Bean userDetailsService

Différents endroits de stockage des mdp



Spring fournit trois implémentations de l'interface UserDetailsService

- ▶ `inMemoryUserDetailsManager(users, ...)`
les login/mdp/roles sont indiqués dans la méthode configure
- ▶ `JdbcUserDetailsManager(dataSource)`
les login/mdp/roles sont stockés dans une BDD
Il faut impérativement lui fournir une `DataSource` préalablement injectée
- ▶ `LdapUserDetailsManager()`
les login/mdp/rôles sont stockés dans un annuaire LDAP

Le Bean userDetailsService

Configurer une authentification inMemory



```
@Bean
public UserDetailsService mesUtilisateurs() {
    UserDetails u1 = User.withDefaultPasswordEncoder()
        .username("phil")
        .password("phil")
        .roles("USER")
        .build();

    UserDetails u2 = User.withDefaultPasswordEncoder()
        .username("paul")
        .password("paul")
        .roles("ADMIN")
        .build();

    return new InMemoryUserDetailsManager(u1, u2);
}
```

`inMemory` est utile quand le nombre d'utilisateurs n'évolue pas.
Un mot de passe en clair dans le code n'est JAMAIS conseillé

Le Bean userDetailsService

Configurer une authentification JDBC



Nécessite de paramétrer la source de données (`application.properties`), créer les tables puis injecter le `DataSource`

```
@Autowired
DataSource dataSource;
@Autowired
PasswordEncoder encoder;
@Bean
public UserDetailsService mesutilisateurs() {
    UserDetails u1 = User.builder()
        .username("phil")
        .password("{bcrypt}$2a$10$7abvS7cYFdBWhRcqZZwgsexMQg00MH8THr...")
        .roles("USER", "ADMIN")
        .build();

    JdbcUserDetailsManager users = new JdbcUserDetailsManager(dataSource);
    users.createUser(u1);
    return users;
}
```

Par défaut : tables `users` (`username, password, enabled`) et `authorities` (`username, authority`) (à créer sauf pour HSQLDB!!)

Le Bean userDetailsService

Il est évidemment possible d'utiliser ses propres tables

```
@Bean
public UserDetailsService mesUtilisateurs() {
    String usersByUsernameQuery = "select username, password, enabled
                                   from t1 where username = ?";

    String authsByUserQuery = "select username, authority
                               from t2 where username = ?";

    JdbcUserDetailsManager users = new JdbcUserDetailsManager(dataSource);

    userDetailsManager.setUsersByUsernameQuery(usersByUsernameQuery);
    userDetailsManager.setAuthoritiesByUsernameQuery(authsByUserQuery);

    // on crée des utilisateurs ici ou dans data.sql
    return users;
}
```

Le Bean userDetailsService

Et pour créer un nouvel utilisateur par programmation

Le JdbcUserDetailsManager offre les méthodes CRUD pour les nouveaux utilisateurs

```
@Bean
public JdbcUserDetailsManager udm() {
    return new JdbcUserDetailsManager(dataSource);
}

List<GrantedAuthority> roles = new ArrayList<GrantedAuthority>();
roles.add(new SimpleGrantedAuthority("ADMIN"));
UserDetails ul = User.withUsername("phil")
    .password("{bcrypt}$2a$10$7abvS7cYFdBWhRcqZZwgsexMQg00MH8THr...")
    .roles("USER", "ADMIN")
    .build();
udm.createUser(user);
```

Le Bean userDetailsService

Récupérer l'utilisateur identifié

Trois techniques :

- Récupérer le token puis le login

```
Authentication auth =
    SecurityContextHolder.getContext().getAuthentication();
String name = auth.getName();
```

- Ajouter des infos en session
implémenter un composant AuthenticationSuccessHandler avec une méthode onAuthenticationSuccess pour ajouter ce que l'on souhaite dans la session
- Injecter l'objet Principal ou l'objet Authentication dans les méthodes des contrôleurs

Plan

Rappels cryptographiques

Spring Security

Le Bean securityFilterChain

Le Bean userDetailsService

Protection contre l'attaque CSRF

Conclusion

Protection contre l'attaque CSRF

Principe

- ▶ SpringSecurity génère à chaque session un token pour éviter les attaques CSRF (Cross-Site Request Forgery).
- ▶ Ce token doit être présent dans chaque requête, Spring le vérifie systématiquement.
- ▶ Cela empêche de ce fait, un formulaire classique du site d'origine ou un site externe (genre H2) d'envoyer des données sans ce token.
- ▶ Pour éviter cette vérification (en TP notamment) :
`.csrf(csrf -> csrf.disable());`

ATTENTION

C'est une solution de facilité, mais une mauvaise solution en situation réelle

Protection contre l'attaque CSRF

CSRF et Formulaires

Si on ne désactive pas la vérification CSRF, chaque formulaire doit impérativement renvoyer le token

```
<form method="post" action="/enregistrer">
<input type="hidden" name="_csrf" value="{_csrf.token}"/>
....
<input type="submit" />
</form>
```

Protection contre l'attaque CSRF

CSRF et H2

- ▶ H2 venant avec son propre serveur, il est vu comme un site externe
- ▶ Si on ne désactive pas le CSRF il faut impérativement autoriser l'url H2, ignorer le token csrf pour cette url, autoriser les frames

```
.requestMatchers(PathRequest.toH2Console()).permitAll()
// pour autoriser l'accès à la page H2
....
.csrf(csrf -> csrf
    .ignoringRequestMatchers(PathRequest.toH2Console()) )
// pour éviter le token CSRF avec H2
....
.headers(headers -> headers
    .frameOptions(frameOptions -> frameOptions.sameOrigin())
// pour les autoriser frames de H2
```

Protection contre l'attaque CSRF

Un exemple complet avec H2 et CSRF

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/activer").hasRole("ADMIN")
            .requestMatchers("/voter").hasAnyRole("ADMIN", "USER")
            .requestMatchers(PathRequest.toH2Console()).permitAll()
            .anyRequest().permitAll()
        )
        .csrf(csrf -> csrf
            .ignoringRequestMatchers(PathRequest.toH2Console())
        )
        .formLogin(Customizer.withDefaults())
        .logout(Customizer.withDefaults())
        .headers(headers -> headers
            .frameOptions(frameOptions -> frameOptions.sameOrigin())
        )
        .build();
}
```

Plan

Rappels cryptographiques

Spring Security

Le Bean `securityFilterChain`

Le Bean `userDetailsService`

Protection contre l'attaque CSRF

Conclusion

Conclusion

En résumé

Une politique de sécurité nécessite de choisir :

- ▶ Quelle page d'authentification ? (`httpBasic`, `formLogin`, perso)
- ▶ Comment je chiffre les mdp ? (`BCrypt`, `Noop`, `Sha-256` ?)
- ▶ Où je stocke les login/mdp ? (mémoire, `jdbc`, `ldap` ?)
- ▶ Quelles pages sont protégées et par quels rôles

SpringBoot offre tous les outils pour cela et notamment :

- ▶ Les Beans `UserDetailsService` et `SecurityFilterChain` permettent de détailler les points précédents.
- ▶ Le `DelegatingPasswordEncoder` qui permet de “jongler” avec les encodages
- ▶ Le Bean `JdbcUserDetailsManager` qui offre le CRUD pour gérer les utilisateurs

Conclusion

Quelques mots sur l'encodage

- ▶ `sha0`, `sha1` `md4`, et `md5` sont “cassés” et ne doivent jamais être utilisés pour le hachage de mots de passe
- ▶ Utilisez plutôt un hachage gourmand en ressources, comme `bcrypt` qui peut être adapté à l'avenir.
- ▶ N'oubliez pas de gérer votre propre sel ! utilisez des méthodes comme `gen_salt()` qui assure des sels uniques à chaque utilisation.
- ▶ Evidemment, n'écrivez pas votre propre crypto, utilisez les outils des spécialistes ;-)